1.i.Create Database

CODING:

```
import java.sql.*;
public class CreateDatabase {
       public static void main(String[] args) {
              try{
              //STEP 2: Register JDBC driver
               Class.forName("com.mysql.cj.jdbc.Driver");
              //STEP 3: Open a connection
              System.out.println("Connecting to database...");
              Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/","root","root
");
              //STEP 4: Execute a query
              System.out.println("Creating database...");
              Statement stmt = con.createStatement();
              stmt.execute("create database BCA");
              System.out.println("Database created successfully...");
              }
              catch(Exception e){e.printStackTrace();}
              }
}
```

STEPS:

EXNO.1 2313721033054

DATE 04/7/25

AIM:

To create database using JDBC program to manipulate student database

STEPS

1 Import packages

Import the java.sql.* package to use JDBC classes

2. Load JDBC Drivers

Load the MySQL driver class using Class.ForName("com.mysql.jdbc.Driver").

3. Establish connection

Use DriverManager.getConnection("jdbc:mysql://localhost:3306/", "root", "password") to connect with MySQL

4 CreateStatement

Create a statement object using con.createStatement().

5. Execute Query

execute the SQL query

6 Close connection

Close the statement and connection objects

OUTPUT:

ii.Create and Alter table

CODING:

```
import java.sql.*;
public class CreateTable {
       public static void main(String args[]){
              try{
              String query;
       Class.forName("com.mysql.cj.jdbc.Driver");
       Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/BCA?useSSL=fal
se","root","root");
       Statement st = con.createStatement();
              query = "CREATE TABLE student_details(id integer,name
text,age integer)";
           st.execute(query);
       System.out.println("Table Created......");
           query = "ALTER TABLE student_details add column course
text";
        st.execute(query);
    System.out.println("Table Altered......");
       con.close();
              }
       catch(Exception e){e.printStackTrace();}
}
}
```

STEPS

2313721033054

EXNO:1

DATE: 04/11/25

Aim

To write a JDBC program to create and alter table

Steps

1 Import packages
Import the java.sql.* package to use JDBC classes

2 Register the Driver
Load and register the JDBC driver using Class.forName() (For MySQL com.mysql.cj.jdbc.Driver)

3. Establish connection
Use
DriverManager.getConnection(url, username, password)
to connect with the database

4 Create statement object
Using con.createStatement() to send SQL queries

5. Execute SQL command to create table
Create table student(ID int primary key, name, age INT); stmt.executeUpdate(query).

6. Execute SQL command to alter table
Alter table student ADD email VARCHAR(100);

7. Close the connection
Close the statement and connection objects to free resources

OUTPUT:

2.i.Insert Record

CODING:

```
import java.sql.*;
public class InsertRecord {
       public static void main(String args[]){
       try
       {
       String query;int t;
       Class.forName("com.mysql.cj.jdbc.Driver");
       Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/BCA?useSSL=fal
se","root","root");
       Statement st = con.createStatement();
       query = "insert into student_details values
(100,'Padma',22,'BCA')";
    t = st.executeUpdate(query);
    query = "insert into student_details values (200,'Jaya',21,'BSC')";
    t = st.executeUpdate(query);
    System.out.println("Records Inserted");
       con.close();
       }
       catch(Exception e){e.printStackTrace();}
   }
}
```

STEPS:

EX NO: 2 2313721033054

DATE 04/7/25

AIM.

To write a JDBC program to insert records into student table

Steps

1 Impost packages

Impost java.sql.* package to use JDBC classes

2. Load JDBC Drives

Load the MySQL drives class using

Class.ForName("com.mysql.cj.jdbc.Drives");

3. Establish Connection

use

DriverManager.getConnection("jdbc:mysql://localhost:3306/student db", "root", "passwrd") to connect with the database.

4. Create statement

Create a statement object using

Con.CreateStatement().

5. Execute Insert Queries

Use executeUpdate() to insert data into the students table

sql

insert into student VALUES(1, 'Arun', 20, 'arun@gmail.com');

6. Close Connection

Close the statement and connection objects

OUTPUT:

II.DISPLAY RECORD

CODING:

```
import java.sql.*;
import java.util.*;
public class DisplayRecord{
public static void main(String args[]){
try{
Scanner sc = new Scanner(System.in);
Class.forName("com.mysql.cj.jdbc.Driver");
Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/BCA?useSSL=fal
se","root","root");
Statement st = con.createStatement();
System.out.println("Enter Table Name:");
String name = sc.next();
ResultSet rs = st.executeQuery("select * from "+name);
while(rs.next())
 System.out.println(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getInt(3)+"
"+rs.getString(4));
con.close();
}
catch(Exception e){
e.printStackTrace();
}
}
}
```

STEPS:

EXNO 2

DATE: 04/11/25

Aim:

To write a JDBC program in Java to display all the records from the student table of the student database

Steps

1 Import packages

import java.sql.* package to use JDBC classes

2 Load JDBC Drivers

Load the MySQL drivers class using Class.forName ("com.mysql.cj.jdbc.Driver");

3 Establish connection

use DriverManager.getConnection("jdbc:mysql://localhost:3306/student db", "root", "password") to connect with the database

4 Create statement

Create a statement object using con.createStatement()

5 Execute select query

execute query using ResultSet rs = stmt.executeQuery("SELECT * FROM students");

6 Display records

print each record to the console

7 Close connection

close the ResultSet, statement, and connection objects

OUTPUT:

3.UPDATE DELETE

CODING:

```
import java.sql.*;
import java.util.*;
public class UpdateDelete{
public static void main(String args[]){
try{
        Scanner sc = new Scanner(System.in);
    Class.forName("com.mysql.cj.jdbc.Driver");
Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/BCA?useSSL=fal
se","root","root");
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("select * from Student_details");
while(rs.next())
System.out.println(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getInt(3)+"
"+rs.getString(4));

System.out.println("Enter the id and age you need to Update:");
int id = sc.nextInt();int age = sc.nextInt();
String query = "update student_details set age = "+age+" where id =
"+id;
int i = st.executeUpdate(query);
System.out.println("Record Updated");

System.out.println("Enter the empid you need to Delete:");
 id = sc.nextInt();
 i = st.executeUpdate("Delete from student_details where id = "+id);
System.out.println(
               "Record Deleted");

con.close();
}
catch(Exception e){
e.printStackTrace();
}
}
}
```

STEPS:

EXNO 3 2313721033054
DATE 11/11/25

AIM.
To JDBC program to update records

Steps.
1. Import packages
Import java.sql.* to use JDBC classes
2. Load the Driver
Use
Class.forName("com.mysql.cj.jdbc.Driver"); to load the JDBC Driver.
3. Establish connection
Use
DriverManager.getConnection(URL, username, password) to connect with the database
4. Prepared Statement
Use PreparedStatement for update query
Ex: UPDATE student set name=? WHERE sno=?
5. Set parameters
(a) pstmt.executeUpdate(); - it returns the number of rows updated
7. Display message
Print whether the record was updated successfully
8. Close connection
Close PreparedStatement and connection object to free resources

OUTPUT:

4.PREPARED STATEMENT

CODING:

```
import java.sql.*;
public class PreparedStatementDemo{
public static void main(String[] args) {
try{
  Class.forName("com.mysql.cj.jdbc.Driver");
  Connection con=DriverManager.getConnection

("jdbc:mysql://localHost:3306/BCA?useSSL=false","root","root");
   PreparedStatement ps;
   ps = con.prepareStatement
                    ("insert into student_details values(?,?,?,?)");
    ps.setInt(1,600);
    ps.setString(2,"Sri");
    ps.setInt(3,18);
    ps.setString(4,"BCom");
    int i = ps.executeUpdate();
    System.out.println (i+"record inserted");
    con.close();
}catch(Exception e)
{
System.out.println(e);
}
}
}
```

STEPS:

EXNO : 4 2313721033054
DATE 11/11/25

Aim:
To program to implement prepared statement interface

Steps:
1. Import packages
Import java.sql.* for JDBC classes

2. Load Drivers
Load mysql drivers using
Class.forName("com.mysql.jdbc.Driver");

3. Establish Connection
Connection con = DriverManager.getConnection(URL, user name, password);

4. Create PreparedStatement
with parameterized SQL query with
example insert into student (sno, name, marks)

5. Set values for parameters
use setters methods like setInt(), setString(),
etc to bind values

6. Execute Update
(ex) pstmt.executeUpdate(), to insert record

7. Display Result
print confirmation message after successful insertion

8. Close Connection
Close PreparedStatement and Connection objects

OUTPUT:

5.BATCH UPDATE

CODING:

```
import java.sql.*;
public class BatchUpdates {
       public static void main(String args[]) throws Exception
       {
              try
              {
              Class.forName("com.mysql.cj.jdbc.Driver");
              Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/BCA?useSSL=fal
se","root","root");
              Statement stmt = con.createStatement();
              stmt.addBatch("insert into student_details values
(800,'Mai',22,'BBA')");
              stmt.addBatch("insert into student_details values
(600,'Rai',23,'MBA')");
              stmt.addBatch("delete from student_details where
id=100");
              stmt.addBatch("update student_details set course = 'MSc'
where id=200");
              stmt.executeBatch();
              System.out.println("Batch command executed");
              con.close();
              }
        catch(Exception e) {}
       }
}
```

STEPS:

EXNO 5 2313721033054
DATE 18/11/25

Aim:
To JDBC program to implementing Batch update

Steps:
1 Import packages
import java.sql.* for JDBC classes

2 Load Drivers
Load JDBC Drivers
Class.forName("com.mysql.cj.jdbc.Driver");

3. Establish Connection
Connection con = DriverManager.getConnection(URL, username, password);

4. Create statement
Statement stmt = con.createStatement();

5. Add Queries to Batch
use stmt.addBatch("SQL query"); to add multiple insert/update/delete queries to the batch

6. Execute Batch
call stmt.executeBatch(); to run all queries in one go.

7 Process Results
The method returns an array of update counts showing number of rows affected by each query.

8. Close Connection
close statement and connection objects.

OUTPUT:

6.SCROLLABLE

CODING:

```
import java.sql.*;
public class Scrollable {
       public static void main(String args[]) {
              try{
                     Class.forName("com.mysql.cj.jdbc.Driver");
                      Connection
con=DriverManager.getConnection("jdbc:mysql://localHost:3306/BCA?useSSL
=false","root","root");
                      Statement
stmt=con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,ResultSet.CO
NCUR_READ_ONLY);
                      ResultSet rs=stmt.executeQuery("select * from
student_details;");
                     System.out.println("records in the table");
                     rs.first();
                     rs.next();
                     rs.previous();
                     rs.last();
                     rs.absolute(2);
                     rs.relative(2);
                     //while(rs.next())
                     System.out.println(rs.getInt(1)+"
"+rs.getString(2)+" "+rs.getInt(3));
                     con.close();
              }catch(Exception e)
               {
                     System.out.println(e); }


              }
       }
```

STEPS:

EXNO:6 2313721033054
DATE.16/9/25

Aim.
To JDBC program to implement scrollable resultset

Steps:
1 Impose packages
Impose java.sql* for JDBC classes

2 Load Drivers
Load mysql drivers:
Class.forName("com.mysql.cj.jdbc.Driver");

3.Establish connection
Use
Driver Manager.getconnection(URL, username, password)
to connect with database

4.Create scrollable statement
Statement stmt = con.createStatement(
ResultSet.TYPE_SCROLL_INSENSITIVE,
ResultSet.CONCUR_READ_ONLY);

5.Execute Query
Use ResultSet rs =
stmt.executeQuery("SELECT * FROM student");

6.navigate using Result set method
rs.next() -> move to next row
rs.previous -> move to previous row
rs.first -> move to first row
rs.last -> move to last row

7.Display Records
Retrieve data from the current row using
rs.getInt(), rs.getString(), etc

8.Close connection
Close Result set, statement, and connection
objects

OUTPUT:

7.LIFECYCLE OF A SERVLET

CODING:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class HelloWorld extends HttpServlet {
       /**
        *
        */
       private static final long serialVersionUID = 1L;
       String message;
    public void init() throws ServletException {
       System.out.println("Start");
         message = "Welcome To BCA";
    }

    public void doGet(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h1>" + message + "</h1>");
    }
    public void destroy()
    {
       System.out.println("Destroy is called");
    }
}
```

STEPS:

EX NO 7 2313721033054
DATE: 25/9/25

Aim:
To program to implement lifecycle of a servlet
steps
1. Create a servlet class
Import required packages
javax.servlet.* and javax.servlet.http.*. create
a class that extends HttpServlet

2 Override init() method
this method is called once when the servlet
is first loaded
Write a print statement to indicate
initialization

3. Override service() method
this method is called every time a request
is made to the servlet
Use PrintWriter to send output to the
client
Display a message showing that the request
is being serviced

4 Override destroy() method
This method is called when the servlet
is about to be unloaded
Write a print statement to indicate
destruction.

5. Configure Deployment Descriptors
(web.xml)
Define the servlet name, class, and URL
mapping.

6. Compile and Deploy
Compile the servlet program
Deploy the .class file in the server's
deployment folder (e.g., tomcat webapps).
Update web.xml

7. Run the application
Start the server (tomcat/glassfish).
Access the servlet in a browser using its URL
mapping.
Observe the console messages
(init, service, destroy).

OUTPUT:

8.

CODING:

Input form.html

```
<html>
<head>
<meta charset="ISO-8859-1">
<title>User Details</title>
</head>
<body >
	<h3>Fill in the Form</h3>

	<form action="InputFormDemo" method="post">
		<table>

			<tr>
				<td>Full Name:</td>
				<td><input type="text" name="name" /></td>
			</tr>
			<tr> <td>Phone Number:</td>
				<td><input type="text" name="phone" /></td>
			</tr>
			<tr>
				<td>Gender:</td>
				<td><input type="radio" name="gender" value="male" />Male
					<input type="radio" name="gender" value="female"
/>Female</td>
			</tr>
			<tr>
				<td>Select Programming Languages to learn:</td>
				<td><input type="checkbox" name="language" value="java"
/>Java
```

```
                                        <input type="checkbox" name="language"
value="python" />Python
                                        <input type="checkbox" name="language" value="sql"
/>SQL
                                        <input type="checkbox" name="language" value="php"
/>PHP</td>
                        </tr>
                        <tr>
                                <td>Select Course duration:</td>
                                <td><select name="duration">
                                        <option value="3months">3 Months</option>
                                        <option value="6months">6 Months</option>
                                        <option value="9months">9
Months</option></select></td>
                        </tr>
                        <tr>
                                <td>Anything else you want to share:</td>
                                <td><textarea rows="5" cols="40"
name="comment"></textarea></td>
                        </tr>

                </table>

                <input type="submit" value="Submit Details">

        </form>

</body>
</html>
```

INPUT FORM DEMO.JAVA

```
import java.io.*;
```

```
import javax.servlet.*;
import javax.servlet.http.*;
public class InputFormDemo extends HttpServlet {
/**
	 *
	 */
	private static final long serialVersionUID = 1L;

protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {

		// Get the values from the request using 'getParameter'
		String name = request.getParameter("name");
		String phNum = request.getParameter("phone");
		String gender = request.getParameter("gender");

		// To get all the values selected for
		// programming language, use 'getParameterValues'
		String progLang[] = request.getParameterValues("language");

		// Iterate through the String array to
		// store the selected values in form of String
		String langSelect = "";
		if(progLang!=null){
for(int i=0;i<progLang.length;i++){
				langSelect= langSelect + progLang[i]+ ", ";
			}
		}

		String courseDur = request.getParameter("duration");
		String comment = request.getParameter("comment");
```

```
        // set the content type of response to 'text/html'
        response.setContentType("text/html");

        // Get the PrintWriter object to write
        // the response to the text-output stream
        PrintWriter out = response.getWriter();

        // Print the data
        out.print("<html><body>");
        out.print("<h3>Details Entered</h3><br/>");

        out.print("Full Name: "+ name + "<br/>");
        out.print("Phone Number: "+ phNum +"<br/>");
        out.print("Gender: "+ gender +"<br/>");
        out.print("Programming languages selected: "+ langSelect +"<br/>");
        out.print("Duration of course: "+ courseDur+"<br/>");
        out.print("Comments: "+ comment);
        out.print("</body></html>");

    }


}
```

STEPS:

EXNO: 8 2313721033054

DATE: 25/9/25

Aim:
To servlet to Read inputs from browser and display output

Steps:
1. Create an HTML Form
Design an html page with a form
Use input elements (like text boxes) to collect user details (e.g name, age).
Set the form action to the servlet URL and and method to get or post

2. Write the servlet program
Import required packages
javax.servlet.*,
javax.servlet.http.*, and
java.io.*
extend HttpServlet class
override doGet() or doPost() method

Use
request.getParameter("name") to fetch input values from the form.

Use printWriter to display output back on the browser

3. Configure Deployment Descriptors
(web.xml)
Define the servlet class, name, and URL mapping.

4. Compile and Deploy
Compile the servlet
Place the class file in the web-INF/classes directory of the web application
Place the HTML form in the web application folder.
Update web.xml with servlet mapping.

5. Run the application
Start the server
Open the HTML form in a browser
Enter details and submit the form
The servlet reads the input and displays the output on the browser.

OUTPUT:

9.SESSION AND HIDDEN FIELD

CODING:

Session.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Session Demo</title>
</head>
<body>
<form action="FirSession">
<center>
<h1> HTTP Session Tracking </h1><br><br>
<h3>Name:<input type="text" name="userName"/></h3></h3><br/><br>
<input type="submit" value="VISIT"/>
</center>
</form>
</body>
</html>
```

Fir session.java

```
import java.io.*;
import javax.servlet.http.*;
public class FirSession extends HttpServlet {
       private static final long serialVersionUID = 1L;
       public void doGet(HttpServletRequest request,
HttpServletResponse response){
        try{

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String n=request.getParameter("userName");

        HttpSession session=request.getSession();
        session.setAttribute("uname",n);

        out.print("<center><form action='SecondSession' method =
'post'>");
        out.print("<h1>HTTP SESSION TRACKING </h1></br></br>");
        out.print("<h2>Welcome " + n+"</h2><br><br>");
        out.print("<input type='submit' value='submit'>");
        out.print("</form>");

        out.close();

                }catch(Exception e){System.out.println(e);}
    }


}
```

Second session.java

```
import java.io.*;
import javax.servlet.http.*;
public class SecondSession extends HttpServlet {
       private static final long serialVersionUID = 1L;
       public void doPost(HttpServletRequest request,
HttpServletResponse response)
       {
               try{

               response.setContentType("text/html");
               PrintWriter out = response.getWriter();

               HttpSession session=request.getSession(false);
               String n=(String)session.getAttribute("uname");
               out.print(" <center> <h3> Hello "+n+" </h3<</center>");

               out.close();

                       }catch(Exception e){System.out.println(e);}
           }


}
```

Hidden fields.html

```
<html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Session Demo</title>
</head>
<body>
<form action="HiddenFieldDemo" method = "post">
<center>
<h1> Hidden Field Demo </h1><br><br>
<h3>Name:<input type="text" name="userName"/></h3></h3><br/><br>
<input type="submit" value="VISIT"/>
</center>
</form>
</body>
</html>
```

HiddenfieldDemo.java

```
import java.io.*;
import javax.servlet.http.*;

public class HiddenFieldDemo extends HttpServlet {

    // class name is FirstServlet which extends HttpServlet
    public void doPost(HttpServletRequest request, HttpServletResponse
response)
    {
        try {
```

```
            response.setContentType("text/html");
            PrintWriter out = response.getWriter();
            String username = request.getParameter("userName");
            out.print("<center><form method = post
action='HiddenFieldDisplay'>");
            out.print("<h1>Hidden Form Field </h1></br></br>");
            out.print("<h2>Welcome " + username+"</h2><br><br>");
            out.print("<input type='hidden' name='username' value='" +
username + "'>");
            out.print("<input type='submit' value='submit'>");
            out.print("</center></form>");
            out.close();
        }
        catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Hidden Field Display.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HiddenFieldDisplay extends HttpServlet {
    public void doPost(HttpServletRequest request, HttpServletResponse
response)
    {
        try {
            response.setContentType("text/html");
            PrintWriter out = response.getWriter();
            String username = request.getParameter("username");
            out.print("<center><h2> Retrieved from Hidden Field
</h2>");
            out.print("<h3> WELCOME " + username+"</h3></center>");
             out.close();
        }
        catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Steps:

EXNO:8 2313721033054

DATE: 25/9/25

Aim:

To servlet to Read inputs from browser and display output.

Steps.

Create an HTML Form

Design an html page with a Form

Use input elements (like text boxes) to collect user details (e.g name, age).

Set the Form action to the servlet URL and and method to get or post.

2. Write the servlet program

Import required packages

javax.servlet.*,

javax.servlet.http.*, and

java.io.*

extend HttpServlet class

Override doGet() or doPost() method

Use

request.getParameter("name") to Fetch input values from the Form.

Use printWriter to display output back on the browser.

OUTPUT:

10.HEADER INFORMATION

CODING:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.*;

public class HeaderDemo extends HttpServlet {
    private static final long serialVersionUID = 1L;
       protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h2>Request Header Information</h2>");
        Enumeration<String> headerNames = request.getHeaderNames();

        while (headerNames.hasMoreElements()) {
            String headerName = headerNames.nextElement();
            String headerValue = request.getHeader(headerName);
            out.println("<b>" + headerName + ":</b> " + headerValue +
"<br>");
        }

        out.println("</body></html>");
    }
}
```

STEPS:

EXNO 10 2313721033054

DATE 11/8/25

Aim. To servlet program to display request headers information

1 Create a servlet class
Impose required package
(javax.servlet.*, javax.servlet.http.*, java.io.*, java.util.*).
Extend HttpServlet
2 Override doGet() method
Set response content type as text/html.
Get the PrintWriter object to send output to the browser.
3 Access request headers
4 Display headers
Iterate through header names using enumeration
Print each header name and its corresponding value in a table or list.
5 Configure Deployment Descriptor
(web.xml).
Define servlet name, class, and URL mapping.
6 Compile, Deploy, and Run
Compile the servlet and place the class file in WEB-INF/classes
Deploy the application on the server (e.g., Tomcat)
Access the servlet URL from a browser and observe request headers details

OUTPUT

11.SERVLET CONTEXT

CODING:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class ServletContextDemo extends HttpServlet{
       private static final long serialVersionUID = 1L;
public void doGet(HttpServletRequest req,HttpServletResponse res)
throws ServletException,IOException
{
res.setContentType("text/html");
PrintWriter pw=res.getWriter();

//creating ServletContext object
ServletContext context=getServletContext();

//Getting the value of the initialization parameter and printing it
String driverName=context.getInitParameter("dname");
String driverpw=context.getInitParameter("dpw");
pw.println("<center><h2>Servlet Context Demo</h2><br><br>");
pw.println("<h3>Will Access the values of Config-Param in
Web.xml</h3>");
pw.println("<h3>Accessible to all Servlets</h3><br><br>");
pw.println("name is="+driverName);
pw.println("pw is="+driverpw);

pw.close();

}}
```

SERVCONTXT.TEXT

```
To be added in web.xml file for ServletContext

   <servlet>
        <servlet-name>ServletContextDemo</servlet-name>
        <servlet-class>ServletContextDemo</servlet-class>
    </servlet>
     <servlet-mapping>
        <servlet-name>ServletContextDemo</servlet-name>
        <url-pattern>/ServletContextDemo</url-pattern>
    </servlet-mapping>
  <context-param>
    <param-name>dname</param-name>
    <param-value>jdbc</param-value>
  </context-param>
   <context-param>
    <param-name>dpw</param-name>
    <param-value>root</param-value>
  </context-param>
```

STEPS:

Exno. 11 2313721033054

Date 11/6/25

Aim:

To implement servlet context

Steps

1. Define a context parameter in web.xml, add a <context-param> entry with a parameter name and value

Example: collegeName = A.R.E engineering college

2. Create the servlet class

Import javax.servlet.*, javax.servlet.http.*, and java.io.*

Extend HttpServlet

3. Access ServletContext object

Use getServletContext() method to obtain the ServletContext object

4. Retrieve context parameters

Use context.getInitParameter("paramName") to read initialization parameters defined in web.xml.

5. Display output

Use PrintWriter to display the retrieved context values on the browser

6 Configure Deployment Descriptors
(web.xml)
Add <servlet>
<servlet-mapping> for the servlet
Add <context-param> for initialization values.

7 Compile, Deploy, and Run
Compile the servlet and place the class
file under WEB-INF/classes
Start the server (Tomcat).
Access servlet URL from a browser and
verify the output.

OUTPUT:

12.SERVLET CONFIG

CODING:

```
import java.io.*;
import java.util.Enumeration;
import javax.servlet.ServletConfig;
import javax.servlet.ServletException;
import javax.servlet.http.*;
public class ServConfigDemo extends HttpServlet{
       public void doGet(HttpServletRequest request,HttpServletResponse
response)
                     throws ServletException,IOException
                     {

           response.setContentType("text/html");
           PrintWriter out = response.getWriter();

           ServletConfig config=getServletConfig();
           Enumeration<String> e=config.getInitParameterNames();

           out.println("<center><h2>Servlet Config Demo</h2><br><br>");
           out.println("<h3>Will Access the values of Init Param in
Web.xml</h3>");
           out.println("<h3>Dedicated to Specific
Servlets</h3><br><br>");

           String str="";
           while(e.hasMoreElements()){
           str=e.nextElement();
           out.print("<br>Parameter Name: "+str);
           out.print("<br>Parameter value:
"+config.getInitParameter(str));
           }

           out.close();
       }
}
```

SERVCONFIG.TEXT

```
To be added in web.xml file for ServletConfig

   <servlet>
        <servlet-name>ServConfigDemo</servlet-name>
        <servlet-class>ServConfigDemo</servlet-class>
         <init-param>
           <param-name>username</param-name>
           <param-value>xyz</param-value>
       </init-param>
        <init-param>
           <param-name>password</param-name>
           <param-value>welcome@123</param-value>
       </init-param>
    </servlet>

     <servlet-mapping>
        <servlet-name>ServConfigDemo</servlet-name>
        <url-pattern>/ServConfigDemo</url-pattern>
```

```
    </servlet-mapping>
```

STEPS:

2313721033054

EXNO.12

DATE 9/05/25

AIM.

To implement servlet config

Steps

1. Define initialization parameters
In web.xml, define <init-param> inside a <servlet> element

2. Create the servlet class
Import javax.servlet.*, javax.servlet.http.*, java.io.*
extend HttpServlet.

3. Access servletConfig object
Use getServletConfig() and servletConfig object

4. Retrieve init parameters
Use
config.getInitParameter("paramName") to
read specific values

5. Display output
Use print writes to display the servlet config
parameters values in the browser

6. Configure deployment descriptors
(web.xml)
Add <init-param> for the servlet in web.xml
map servlet name and URL pattern.

7. Compile, Deploy and Run
Compile the servlet and place the class
file in WEB-INF/classes.
Start the server (eg. Tomcat)
Access servlet URL from a browser to
verify output

OUTPUT:

13.

Employee.html

```
<html>
<head>
<title>Employee Details</title>
</head>
<body>
<form>
<h2 align= "center"> Employee Registration Form </h2> <br>

<table align = "center">
  <tr><td><input type = "button" value = "Insert" onclick =
"window.location.href = 'insertform.html'"/></td>
  <td><input type = "button" value = "Display" onclick =
"window.location.href = 'Display'"/></td>

</table>

</form>
</body>
</html>
```

Display.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.sql.*;

public class Display extends HttpServlet {

       /**
        *
        */
       private static final long serialVersionUID = 1L;

       protected void doGet(HttpServletRequest request,
HttpServletResponse response)
                     throws ServletException, IOException {
               response.setContentType("text/html;charset=UTF-8");
               PrintWriter out = response.getWriter();
try
               {
               Class.forName("com.mysql.cj.jdbc.Driver");
               Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/new?useSSL=fal
se","root","root");
               Statement st = con.createStatement();
               ResultSet rs = st.executeQuery("select * from EmpInfo");

               RequestDispatcher rd =
request.getRequestDispatcher("/Employee.html");
               rd.include(request, response);
               out.println("<center><table border = 2 width = 500>");
               out.println("<tr><th>Emp ID</th><th>Name</th>");
               out.println("<th> Age</th><th>Designation</th>");
                     while(rs.next())
```

```
                {
                 out.println("<tr><td>"+rs.getInt(1)+"</td>");
                 out.println("<td>"+rs.getString(2)+"</td>");
                 out.println("<td>"+rs.getInt(3)+"</td>");
                 out.println("<td>"+rs.getString(4)+"</td>");
                }
                con.close();
                out.println("</table></center>");

  }
      catch(Exception se) {se.printStackTrace();}


       }
}
```

Web.xml

```
  <?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd" id="WebApp_ID"
version="4.0">
  <display-name>ServletDemos</display-name>
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
    <welcome-file>index.jsp</welcome-file>
    <welcome-file>index.htm</welcome-file>
    <welcome-file>default.html</welcome-file>
    <welcome-file>default.jsp</welcome-file>
    <welcome-file>default.htm</welcome-file>
  </welcome-file-list>
   <servlet>
<servlet-name>Insert</servlet-name>
        <servlet-class>Insert</servlet-class>
    </servlet>
     <servlet-mapping>
        <servlet-name>Insert</servlet-name>
        <url-pattern>/Insert</url-pattern>
    </servlet-mapping>
   <servlet>
<servlet-name>Insert</servlet-name>
        <servlet-class>Insert</servlet-class>
    </servlet>
     <servlet-mapping>
        <servlet-name>Insert</servlet-name>
        <url-pattern>/Insert</url-pattern>
    </servlet-mapping>
   <servlet>
```

InsertForm.html

```
!DOCTYPE html>
<html>
<head>
<title>Record Insertion</title>
</head>
<body>
<form method = "post" action = "Insert">
<h2 align = "center">Adding Records to Database</h2>
<center>
```

```
<table>
<tr><td>EmpId</td><td><input type = "text" name="id"></td>
<tr><td>Name</td><td><input type = "text" name="name"></td>
<tr><td>Age</td><td><input type = "text" name="age"></td>
<tr><td>Designation</td><td><input type = "text" name="desig"></td>
<tr><td colspan=2 align = "center"><input type = "submit" value =
"Insert"></td>
</table>
<a href = "Employee.html">Back..</a>
</center>

</form>
</body>
</html>
Insert.java
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.sql.*;

public class Insert extends HttpServlet {

	private static final long serialVersionUID = 1L;

	protected void doPost(HttpServletRequest request,
HttpServletResponse response)
			throws ServletException, IOException {
		response.setContentType("text/html;charset=UTF-8");
		PrintWriter out = response.getWriter();

		Integer id =
Integer.parseInt(request.getParameter("id"));
		String name = request.getParameter("name");
		Integer age =
Integer.parseInt(request.getParameter("age"));
		String desig = request.getParameter("desig");
try
		{
		Class.forName("com.mysql.cj.jdbc.Driver");
		Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/new?useSSL=fal
se","root","root");
		PreparedStatement ps = con.prepareStatement
				("insert into EmpInfo values(?,?,?,?)");
ps.setInt(1, id);
	ps.setString(2, name);
	ps.setInt(3, age);
	ps.setString(4, desig);
	int i = ps.executeUpdate();
	RequestDispatcher rd =
request.getRequestDispatcher("/Employee.html");
	rd.include(request, response);
if(i > 0)
		out.println("<H4 align = center> You are successfully
registered <H4>");
	else
		out.println("<H4 align = center> Error in Registration
<H4>");
```

```
}
catch(Exception se) {
        se.printStackTrace();
}



        }
}
```

EXNO 13

DATE 29/8/25

Aim.

To servlet to manipulate employee database

steps.

1 create database

open MySQL and create a database named employeeDB

create a table employee with fields id, name, department and salary

2 set up project

create a Dynamic web project in Eclipse

Add mysql-connector-java jar to the project WEB-INF/lib folder

3 create Database connection class

create a class named DBconnection.java

write code to establish connection with the employee DB using JDBC

4. create servlet

create a servlet named employeeServlet.java

write code to handle the HTTP POST request

insert employee details into the database using prepared statement

Output:

14.
Coding:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
   pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
   <title>Arithmetic Functions in JSP</title>
</head>
<body>
<%!
   int a = 20;
   int b = 10;
   int sum = a + b;
   int difference = a - b;
   int product = a * b;
   int quotient = a / b;
   int modulus = a % b;
%>

<h2>Arithmetic Operations in JSP (Scriptlet)</h2>
<p>Addition: <%= sum %></p>
<p>Subtraction: <%= difference %></p>
<p>Multiplication: <%= product %></p>
<p>Division: <%= quotient %></p>
<p>Modulus: <%= modulus %></p>

</body>
</html>
```

STEPS:

EXPT. 14 2313721033054
DATE: 29/8/25

AIM:
To arithmetic functions in JSP

STEPS:
1. Create a JSP File
Open your Java IDE as text editor
Create a new file with the JSP ex extension
2. Write HTML structure
Add the standard HTML tags like <html>, <head> <body>
3. Add input form
Create a form using <form> tag
Add field to accept two numbers from the user.
add options as button to choose the arithmetic operation
4. Retrieve input values in JSP
Use request.getParameter() to get the values entered by the user
5. Perform arithmetic operation
Write JSP scriptlets to perform addition, subtraction multiplication, and division
Store the result in a variable
6. Display the Result
Use out.print() as expression tags <= result />
to display the result on the web page
7. Test the application
Run the JSP file on a server like Apache Tomcat
enter numbers and select an operation
submit the form and check if the result is displayed correctly

OUTPUT:

15.
CODING:

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
   pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
   <meta charset="UTF-8">
   <title>JSP Method Declaration Example</title>
</head>
<body>
   <h2>JSP Method Declaration Example</h2>

   <%
      // Call the method and store the result
      String message = greetUser("Yasika");
   %>

   <p><%= message %></p>

   <hr>

   <%
      int num1 = 10;
      int num2 = 20;
      int sum = addNumbers(num1, num2);
   %>

   <p>Sum of <%= num1 %> and <%= num2 %> is <%= sum %>.</p>

   <!-- Method Declaration -->
   <%!
      // A method to greet the user
      public String greetUser(String name) {
         return "Hello, " + name + "! Welcome to JSP.";
      }

      // A method to add two numbers
      public int addNumbers(int a, int b) {
         return a + b;
      }
   %>
</body>
</html>
```

STEPS:

EXNO:15 2313721033054

DATE.2018/25

AIM:

To JSP method declaration

STEPS:

1. Create a JSP File

Open your Java IDE or editor

Create a new File . JSP extension

2. Add HTML structure

Write the basic HTML tags such as <html>,<head>,

<body>.

3 Declare JSP methods

Use the <%!... %> directive to declare method

Write your method with a return type,

name, and parameters inside this block

example

JSP

<%!

int add(int a, int b) {

return a+b;

}

%>

4 Invoke the method

Inside <% .... %> scriptlet, call the method by

passing required parameters

Store the return value in a variable if

needed

5 Display the result

Use out.print() or expression tag <%= ... %> to

display the result on the webpage.

6. Run the JSP page

Deploy the JSP File on a server like Apache

tomcat

OUTPUT:

16.
CODING:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
   pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<body>
<%-- JSP comments --%>
<%@page import="java.util.Date"%>
<%!
   Date date = new Date();
%>
<b>System date and time: </b> <%= date %>

</body>
</html>
```

STEPS:

EXNO:16 2313721033054

DATE 30/8/25

Aim:

To JSP page to print current date and time implementing directives

Steps

1. Create a JSP file
Open the Java IDE or text editor

2. Use the page directive to import classes

3. Write the HTML structure
Add the basic HTML tags <html>, <head>, <body>

4. Create an object of Date class
Inside the JSP scriptlet

JSP

```
<%
    Date date = new Date();
%>
```

5. Display the current Date and time
Use out.println as expression tags <%= date %>
to print the date and time on the web page

6. Run the JSP page
Start the apache tomcat server
Access the JSP file through the browser
Check that the current date and time are displayed

OUTPUT:

17.

```
package com.example;

public class MyUtility {

   // A simple method to return a greeting message
   public String getGreeting(String name) {
      return "Hello, " + name + "! Welcome to JSP.";
   }
}
```

```
<%@ page import="com.example.MyUtility" %>
<!DOCTYPE html>
<html>
<head>
   <title>JSP Include Java Method Example</title>
</head>
<body>
   <h2>JSP Method from Java Class Example</h2>
   <%
      // Create an instance of the Java class
      MyUtility utility = new MyUtility();

      // Call the method with a name
      String message = utility.getGreeting("Yasika");
   %>
   <p><%= message %></p>
</body>
</html>
```

STEPS:

2313721033054

EX NO. 17

DATE :

AIM

To demonstrate how to reuse java class methods in a JSP page

STEPS:

1 create a java class with a method
2. compile the java class and place it in the WEB-INF / classes folder
3. IN JSP, use the import directive to include the class
4. create an object of the class in JSP
5. call the method and process the result
6. Display output on the browser
7. Deploy and test

OUTPUT:

18.

CODING:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
   pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
 <form method="post">
 <h2>Simple Calculator</h2>
  Enter the Value for A: <input type="text" name="t1"><br>
  Enter the Value for B: <input type="text" name="t2"><br>
  Operation: <select name="operation">
  <option value="1">+</option>
  <option value="2">-</option>
  <option value="3">*</option>
  <option value="4">/</option>
  <option value="5">%</option>
</select>    
  <input type="submit" value="Submit">
</form>
   <%
   String s1 = request.getParameter("t1");
   String s2 = request.getParameter("t2");
   if(s1 != null && s2 != null){
      int a = Integer.parseInt(s1);
      int b = Integer.parseInt(s2);
      int ans=0;

    String s = request.getParameter("operation");
    switch(s) {
case "1":
    {
      out.print("Addition = "+(a+b));
      break;
    }
    case "2":
    {
       out.print("Subtraction = "+(a-b));
       break;
    }
    case "3":
    {
            out.print("Multiplication = "+(a*b));
       break;
    }
    case "4":
    {
            out.print("Division = "+(a/b));
```

```
      break;
   }
   case "5":
   {   out.print("Remainder= "+(a%b));
      break;}
   }

  }
%>

</body>
</html>
```

STEPS:

EXNO 18 2313721033054

DATE

AIM

To create a JSP program that functions as a simple calculator

STEPS.

1 Create a Html form to accept two numbers and an operator

2 Use JSP to read form inputs

3. Based on the operator, perform addition, subtraction, multiplication or division

4. Display the calculated result

5. Handle invalid inputs gracefully

6 Save, deploy and test the calculator

7 Verify with multiple operations

OUTPUT:

19.
CODING:

```
%@ page language="java" contentType="text/html; charset=ISO-8859-1"
   pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<form method = "post">
  <h2> This is an example for Multiplication Table</h2>
  <input type = "text" name = "t1">
  <input type = "submit" value = "OK">
   <br/><hr/>
  </form>
 <%
 String s =  request.getParameter("t1");
 if (s!=null){
  int n = Integer.parseInt(s);%>
<br><hr/><h3>The multiplication table of <%= n%>  </h3>
 <% for(int i =1; i<=10;i++){
 out.print(n+" x "+i+"= "+n*i+"<br>");
 } }%>

</html>
```

STEPS:

EX NO. 14

DATE. 12/9/25

Aim

To write a JSP program that generates and displays multiplication tables

Steps.

1 Create a JSP File

2 Accept a number from the user using an html form

3 Read the input in JSP using request parameters

4 Use a loop to generate the multiplication table

5. Print the table in HTML format

6. Deploy and run on the server

7 Verify by entering different numbers

OUTPUT:

20.
CODING:
Cookies.html

```
<html>
<head>
<meta charset="ISO-8859-1">
<title>Session Demo</title>
</head>
<body>
<form action="CookieDemo" method = "post">
<center>
<h1> Cookie Demo </h1><br><br>
<h3>Name:<input type="text" name="userName"/></h3></h3><br/><br>
<input type="submit" value="VISIT"/>
</center>
</form>
</body>
</html>
```

Cookie Demo.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class CookieDemo extends HttpServlet {

  public void doPost(HttpServletRequest request, HttpServletResponse
response){
    try{

    response.setContentType("text/html");
    PrintWriter out = response.getWriter();

    String n=request.getParameter("userName");
Cookie a=new Cookie("uname",n);//creating cookie object
    Cookie b = new Cookie("pw","123");
    out.print("<center><h1>Welcome "+n+"</h1></center>");
    response.addCookie(a);
    response.addCookie(b);//adding cookie in the response

    //creating submit button
    out.print("<center><form action='CookieServlet' method = 'post'>");
    out.print("<input type='submit' value='ShowCookie'>");
    out.print("</form></center>");

    out.close();

        }catch(Exception e){System.out.println(e);}
  }
}
```

Cookie Servlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class CookieServlet extends HttpServlet {
```

```
public void doPost(HttpServletRequest request, HttpServletResponse
response){
    try{

response.setContentType("text/html");
    PrintWriter out = response.getWriter();

    Cookie ck[]=request.getCookies();
    out.println("<center><h2>Retrieving Data From Cookie</h2><br><br>"+
    "<h3>User Id : "+ck[0].getValue()+"<br>"+
    "Password:"+ck[1].getValue()+"</h3></center>");

    out.close();

         }catch(Exception e){System.out.println(e);}
    }
}
```

STEPS:

EXNO 20 2313721033054
DATE 12/9/25

AIM.
To demonstrate the use of cookies in JSP for storing and retrieving user information

Steps.
1. Create a JSP page
2. Accept user input from a form
3. Create a cookie object with key-value pairs
4 Add the cookie to the response
5 Retrieve cookies from the request in another JSP page
6. Display stored cookie information
7. Test by running in a browser

OUTPUT:

21.
CODING:

```
<html>
<head>
    <title>Chemistry Element Search</title>
</head>
<body>
    <h2>Search Chemistry Element</h2>
    <form method="post">
        Enter Element Symbol (H, O, C, Na, Cl, Fe):
        <input type="text" name="element" required>
        <input type="submit" value="Search">
        <hr>
    </form>

    <%
String element = request.getParameter("element");
        if (element != null) {
            element = element.trim();
if (element.equalsIgnoreCase("H")) {
                out.println("<h3>Hydrogen: The lightest element, atomic
number 1.</h3>");
            } else if (element.equalsIgnoreCase("O")) {
                out.println("<h3>Oxygen: Essential for respiration,
atomic number 8.</h3>");
            } else if (element.equalsIgnoreCase("C")) {
                out.println("<h3>Carbon: Basis of organic chemistry,
atomic number 6.</h3>");
            } else if (element.equalsIgnoreCase("Na")) {
                out.println("<h3>Sodium: Reactive alkali metal, atomic
number 11.</h3>");
            } else if (element.equalsIgnoreCase("Cl")) {
  out.println("<h3>Chlorine: A halogen, used in disinfectants, atomic
number 17.</h3>");
            } else if (element.equalsIgnoreCase("Fe")) {
                out.println("<h3>Iron: Transition metal, important in
steel making, atomic number 26.</h3>");
            } else {
                out.println("<h3>No details found for \"" + element +
"\"</h3>");
            }
        }
    %>
</body>
</html>
```

STEPS:

EX No: 21 2313721033054

DATE: 12/9/25

Aim:

To implement a JSP application that searches and display definitions of chemistry elements

Steps

1. Create an Html Form to accept element name
2. Pass the input to a JSP page
3. Store element details in a database as collection
4. Select for the element in the data source
5. Display the definition if found, else show a message
6. Deploy and test with different input
7. Ensure results are correctly displayed

OUTPUT:

22.

CODING:
FORM.HTML

```
<!DOCTYPE html>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>User Details</title>
</head>
<body >
       <h3>Fill in the Form</h3>

       <form action="DisplayForm.jsp" method="post">
              <table>
<tr>
                            <td>Full Name:</td>
                            <td><input type="text" name="name" /></td>
                     </tr>
                     <tr>
                            <td>Phone Number:</td>
                            <td><input type="text" name="phone"
/></td>
                     </tr>
                     <tr>
                            <td>Gender:</td>
                            <td><input type="radio" name="gender"
value="male" />Male
                                   <input type="radio" name="gender"
value="female" />Female</td>
                     </tr>
                     <tr>
<td>Select Programming Languages to learn:</td>
                            <td><input type="checkbox" name="language"
value="java" />Java
                                   <input type="checkbox"
name="language" value="python" />Python
                                   <input type="checkbox"
name="language" value="sql" />SQL
                                   <input type="checkbox"
name="language" value="php" />PHP</td>
                     </tr>
                     <tr>
                            <td>Select Course duration:</td>
                            <td><select name="duration">
                                   <option value="3months">3
Months</option>
                                   <option value="6months">6
Months</option>
                                   <option value="9months">9
Months</option></select></td>
                     </tr>
                     <tr>
                            <td>Anything else you want to share:</td>
                            <td><textarea rows="5" cols="40"
name="comment"></textarea></td>
                     </tr>
</table>
```

```
                <input type="submit" value="Submit Details">

        </form>

</body>
</html>
```

DISPLAY FORM.JSP

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-
1">
<title>Insert title here</title>
</head>
<body>
<h2> STUDENT REGISTRATION DETAILS</h2>
<hr>
Student Name = <%= request.getParameter("name") %><br>
Phone Number = <%= request.getParameter("phone") %> <br>
Gender = <%= request.getParameter("gender") %><br>
<%String progLang[] = request.getParameterValues("language");
 String langSelect = "";
if(progLang!=null){
                      for(int i=0;i<progLang.length;i++){
                              langSelect= langSelect + progLang[i]+ "|
";
                      }
               }
               out.print(langSelect);%>

               <br>

Course Duration = <%=request.getParameter("duration")%> <br>
Comments = <%= request.getParameter("comment")%>


</body>
</html>
```

STEPS:

EXNO 22 2313721033054

DATE 26/9/25

AIM.

To demonstrate the use of Form in JSP For collecting and processing user input

Steps.

1. Create an HTML Form with input Fields
2. Submit the Form data to a JSP page
3. Retrieve Form data using request.getParameter()
4. Process the input values in JSP
5. Display the result back on the browser
6. Deploy and test the application
7. verify correct data handling

OUTPUT:

23.

CODING:
DATA FORM TEXT.HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>AJAX Read TXT</title>
</head>
<body>
    <h2>Retrieve Data from TXT file using XMLHttpRequest</h2>
    <button onclick="loadText()">Get Data</button>
    <p id="output"></p>
<script>
        function loadText() {
            var xhr = new XMLHttpRequest();
            xhr.open("GET", "data.txt", true);
            xhr.onreadystatechange = function () {
                if (xhr.readyState === 4 && xhr.status === 200) {
                    document.getElementById("output").innerText =
xhr.responseText;
                }
            };
            xhr.send();
        }
    </script>
</body>
</html>
```

STEPS:

EXNO:23 2313721033054

DATE 26/9/25

Aim.

To demonstrate Ajax using XML Http Request to Fetch and display data from a text File without reloading the page

Steps

1 Create an Html page with a button

2 Write javascript to create an XMl Http Request object

3 open a request to a text File

4. send the request and wait for response

5 Display the response content in the browser

6 save and deploy the program

7. test dynamic loading without refreshing

OUTPUT:

24.

CODING:

```
<!DOCTYPE html>
<html>
<head>
    <title>AJAX - Retrieve Header Information</title>
</head>
<body>
    <h2>Retrieve Header Information of a File</h2>
    <button onclick="getHeaders()">Get Header Info</button>
    <div id="output"></div>
<script>
        function getHeaders() {
            var xhr = new XMLHttpRequest();
            xhr.open("HEAD", "data.txt", true);  // HEAD request gets
only headers
            xhr.onreadystatechange = function () {
                if (xhr.readyState == 4 && xhr.status == 200) {
                    var headers = "";
                    headers += "Content-Type: " +
xhr.getResponseHeader("Content-Type") + "<br>";
                    headers += "Content-Length: " +
xhr.getResponseHeader("Content-Length") + "<br>";
                    headers += "Last-Modified: " +
xhr.getResponseHeader("Last-Modified") + "<br>";
                    document.getElementById("output").innerHTML =
headers;
                }
            };
            xhr.send();
  }
    </script>
</body>
</html>
```

STEPS:

EXNO 24 2313721033054

DATE 26/4/25

Aim

To write a program that retrieves and display headers information of a resource file using JSP/Servlet

Steps

1 Create a servlet or JSP page

2 Use URL connection class to connect to a resource file

3 Retrieve headers using getHeaderFields() or similar method

4 Store the retrieved headers information

5 Display the headers in the browser

6 Deploy and run on the server

7 Verify headers details like content type and length

OUTPUT:

25.
CODING:

```
<!DOCTYPE html>
<html>
<head>
    <title>AJAX - Retrieve Specific Header</title>
</head>
<body>
    <h2>Retrieve Specific Header Information</h2>
    <button onclick="getHeaderInfo()">Get Last-Modified</button>
    <p id="output"></p>
<script>
        function getHeaderInfo() {
            var xhr = new XMLHttpRequest();
            xhr.open("HEAD", "data.txt", true);   // HEAD request →
headers only
            xhr.onreadystatechange = function () {
                if (xhr.readyState == 4 && xhr.status == 200) {
                    // Retrieve specific header
                    var lastModified = xhr.getResponseHeader("Last-
Modified");
                    document.getElementById("output").innerHTML =
                        "Last-Modified: " + lastModified;
                }
            };
            xhr.send();
  }
    </script>
</body>
</html>
```

STEPS:

2313721033054

EXNO 25

DATE 26/9/25

Aim.

To write a program that retrieves and displays specific header information of a resource using JSP/servlet

Steps

1. Create a JSP page or servlet program
2. Use URL connection or HttpServletRequest to access the resource
3. Call method like getHeaderField("content-type") to fetch specific headers
4. Store the retrieved header values
5. Display the selected header information in the browser
6. Deploy and test on the server
7. Verify by retrieving different header fields

OUTPUT: